A CLIC Extension Based Fast Interrupt System for Embedded RISC-V Processors

2021 6th International Conference on Integrated Circuits and Microsystems (ICICM) | 978-1-6654-5886-3/21/$31.00 ©2021 IEEE | DOI: 10.1109/ICICM54364.2021.9660345

Binjie Mao
School of Micro-Nano Electronics
Zhejiang University
Hangzhou, China
maobinjie1996@zju.edu.cn

Nianxiong Tan
School of Micro-Nano Electronics
Zhejiang University
Hangzhou Vango Technologies, Inc.
Hangzhou, China
ntan@zju.edu.cn

Ting Chong
State Grid Key Laboratory of Power Industrial Chip Design and Analysis Technology
Beijing Smart-Chip Microelectronics Technology Co., Ltd
Beijing, China
chongting@sgitg.sgcc.com.cn

Lei Li
State Grid Key Laboratory of Power Industrial Chip Design and Analysis Technology
Beijing Smart-Chip Microelectronics Technology Co., Ltd
Beijing, China
lilei2@sgitg.sgcc.com.cn

***Abstract*—A fast interrupt response time is a very important feature for embedded processors. In this paper, we design a fast interrupt system for embedded RISC-V processors. It is further extended on the Core-Local Interrupt Controller (CLIC) based interrupt system. When an interrupt happens, the general-purpose registers and other interrupt related registers such as Control and Status Registers (CSRs) are pushed to the stack memory automatically. Thus, there is no additional software overhead in the Interrupt Service Routine (ISR), and the real handler code can be executed immediately. The interrupt response time of the interrupt system based on the CLIC extension is shorter than that of a RISC-V processor only employing the CLIC, and is comparable to that of an ARM Cortex-M3 processor.**

Keywords—RISC-V, interrupt system, embedded processor, CLIC

I. INTRODUCTION

The interrupt response time is defined as the time it takes from the assertion of the interrupt request by the interrupt device, to the start of the execution of the first task instruction in the Interrupt Service Routine (ISR). It is one of the key characteristics of a micro-controller and is essential for many real-time embedded applications. Nowadays, ARM processors dominate the embedded market. The Arm processor is a mature commercial instruction set architecture (ISA). The Cortex-M processors are tailor-made by ARM for low-power & micro-control applications. The Nested Vector Interrupt Controller (NVIC) is the critical part of the interrupt system in the Cortex-M processors [1]. It enables automatic nested interrupt support so that the interrupt latency of all the Cortex-M processors is low. However, the commercial ISA is a closed system and it doesn't support customizing the processor according to the needs. As a completely open-source ISA, RISC-V [2] is becoming more widely used among IC design companies. For embedded applications, the RISC-V specification adds a fast interrupt proposal named Core-Local Interrupt Controller (CLIC) [3]. Compared with the commonly used Platform-Level Interrupt Controller (PLIC) [4] of RISC-V, the CLIC simplifies the control logic of the interrupt response and uses new interrupt-related Control and Status Registers (CSRs) [2] for the nesting of interrupts. However, the general CLIC based interrupt system is still accompanied by software overhead to support nested interrupt. Therefore, the interrupt response time of the CLIC based interrupt is still longer than that of the Cortex-M processors.

RISC-V, as an open-source ISA, is of interest because one can tail-make his own processor making the design self-sufficient. Therefore, designing an interrupt system applied to embedded RISC-V processors that can match the ARM Cortex-M processors in terms of the interrupt response time has significance of research and application. This paper studies the process of ARM Cortex-M processors (taking Cortex-M3 as an example) and SiFive-E2 processors (taking E21 as an example) entering the interrupt handler and compares the interrupt response time of the two interrupt systems. By using the hardware stacking mechanism similar to that of ARM, we design a fast interrupt system to reduce the interrupt response time and software overhead. The system is based on the CLIC extension, supporting automatic nested interrupt. The processor automatically saves the context once an interrupt happens and is able to directly execute the real handler code after entering the handler without the involvement of software.

II. EXISTING SCHEME ANALYSIS

In the description of the interrupt schemes below, we mainly analyze the interrupt entry sequence of ARM Cortex-M3 [5] and SiFive-E21 [6]. We concentrate on the time taken from the interrupt occurrence to the execution of the real handler code.

A. Cortex-M3 Interrupt Entry Sequence

The interrupt system of Cortex-M3 contains the M3 core and the NVIC. The M3 core implements a three-stage pipeline and the NVIC design supports up to 240 interrupt inputs plus a number of built-in system exceptions.

When an interrupt is accepted on the Cortex-M3 processor, some of the registers in the register banks (R0 to R3, R12, R14), the return address program counter (PC), and the Program Status Register (xPSR) are pushed to the current active stack memory automatically. The Link Register

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on September 29,2026 at 07:10:13 UTC from IEEE Xplore. Restrictions apply.

(LR/R14) is then updated to a special value to be used during interrupt return. The handler starting address is fetched by the processor, and then the interrupt handler starts to execute the real handler code without software overhead. The interrupt entry sequence of Cortex-M3 is shown in Fig. 1.

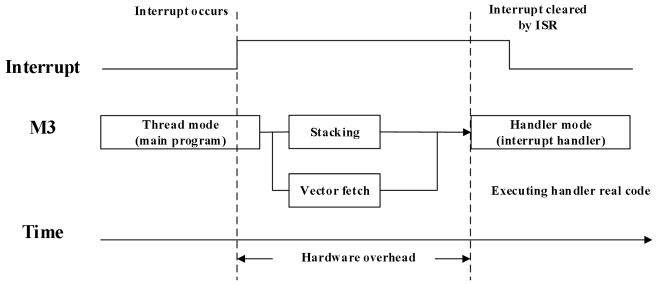


Fig. 1. Interrupt entry sequence of Cortex-M3.

B. E21 Interrupt Entry Sequence

The interrupt system of SiFive's RISC-V processor E21 [6] contains a RISC-V core and the CLIC. The E21 core implements three-stage pipeline. The CLIC is backward compatible with the Core-Local Interrupt (CLINT) [4] for software, timer, and external interrupts. The timer inside the RISC-V processor corresponds to the SysTick of the Cortex-M3. In addition, the CLIC of E21 also supports 127 core-local interrupts, which are directly connected to the interrupt sources.

When an interrupt is accepted on the E21 processor in the CLIC mode, the interrupt-related Control and Status Registers (CSRs) updates. The function of these registers is shown in Table I. The MIE field of machine status (mstatus) register is copied into MPIE field of machine cause (mcause) register, and then the MIE field is cleared, effectively disabling interrupts. The interrupted interrupt level is copied into MPIL field of mcause register, and the interrupt level is set to that of the incoming interrupt. The privilege mode prior to the interrupt is encoded in MPP field of mcause register. The current PC is copied into the machine exception program counter (mepc) register, and then PC is set to the value fetched from the vector table. Afterwards, E21 enters the interrupt handler but the interrupt service cannot start yet. The general-purpose registers, the mstatus register, the mcause register, and the mepc register need to be pushed into stack memory, and the processor sets the MIE to prepare for preemption. Then, E21 executes the first handler real code. The interrupt entry sequence of SiFive-E21 is shown in Fig. 2.

TABLE I. INTERRUPT-RELATED CSRS IN CLIC MODE

Register Name	Field	Description
Machine status	MIE	Machine interrupt enable.
Machine cause	MPIE	Machine previous interrupt enable. (CLIC mode only)
	MPIL	Previous interrupt level. (CLIC mode only)
	MPP	Previous interrupt privilege mode. (CLIC mode only)

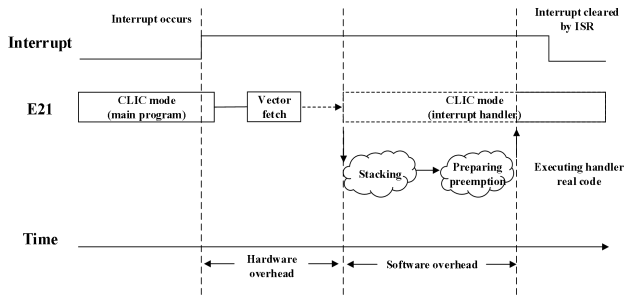


Fig. 2. Interrupt entry sequence of E21.

C. Comparison

Typically, we define the time from the assertion of the interrupt request to the execution of the first instruction of the interrupt service routine (ISR) as the processor's interrupt latency. It refers to the hardware overhead of the processor responding to an interrupt. In ARM Cortex-M processors, all the interrupt response related work is done by hardware, so the interrupt response time of M3 is hardware overhead only. However, in the RISC-V architecture, there is no specific design for automatic stacking. Therefore, the context needs to be saved first by software before the processor enters the interrupt handler. Secondly, the global-enable bit will be hardwired to zero once the interrupt be responded in an RISC-V processor. The processor needs to reset the global-enable bit and saves the PC return value, the current interrupt level, and the current interrupt cause to support interrupt preemption in the handler. After that, the real handler code is actually executed. The operations mentioned above are all software overhead. Therefore for the RISC-V processors, the complete interrupt response time includes hardware and software overhead.

In order to compare the interrupt response time of ARM Cortex-M3 and SiFive-E21 objectively, we have made the following assumptions:

- The memory system has zero wait state (and with resources not being used by other bus masters).
- The interrupt service is not being blocked by another current running exception/interrupt service.
- Stack 8 registers in all.
- Support the nested interrupt and use the same form of the vectored interrupt.

TABLE II. INTERRUPT RESPONSE TIME COMPARISON OF M3 AND E21

Processor	Cortex-M3	SiFive-E21		
Overhead	Hardware overhead	Hardware overhead	Software stacking	Software preemption preparation
Time (cycle)	12	6	9	5
Total (cycle)	12	20		

The comparison of interrupt response time is indicated in Table II.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on September 29,2026 at 07:10:13 UTC from IEEE Xplore. Restrictions apply.

III. DESIGN OF THE INTERRUPT SYSTEM

The design of the interrupt system is not only related to the interrupt controller, but also related to the micro-architecture design of the processor. The processors based on Harvard architecture [8] can support simultaneous instruction and data access, such as Cortex-M3 and E21 mentioned above. In order to achieve a lower interrupt response time, we design this interrupt system based on a RISC-V processor with a three-stage pipeline [9]. The processor is based on RV32-E with the CLIC vectored mode [7].

A. System Architecture

Fig. 3 shows the architecture of the interrupt system. The system is based on the general design of the RISC-V interrupt system using the CLIC and extends the design of interrupt-related control inside the processor. The system contains a RISC-V core and the CLIC. The CLIC is responsible for sampling and arbitrating the signal from the interrupt sources and sending the results to the RISC-V core. The core is responsible for the hardware stacking and vector fetching. The system utilizes the Harvard architecture. Thus the processor can have a simultaneous instruction and data access with the AHB bus.

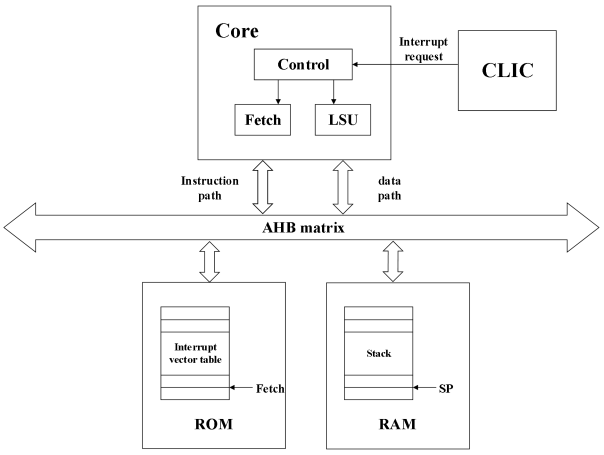


Fig. 3. Architecture of the interrupt system.

B. Interrupt Controller Design

The design of CLIC includes three parts: the interrupt request generator, the preemption support unit, and the interrupt vector table.

1) Interrupt request generator: This part is the main body of the CLIC. It deals with the interrupt request from 32 core local external interrupt sources and arbitrates the result to the RISC-V core. Fig. 4 shows the logic structure of the interrupt request generator. Each interrupt source has its Interrupt Pending (IP), Interrupt Enable (IE), and Interrupt Level (IL) register. They are both readable and writable. When the module detects the positive edge of an interrupt signal, the IP register will be set to 1. It means that the interrupt source is pending for response. The IP register will be cleared once the ISR of the interrupt source being executed. In addition, users can configure the interrupt-enable and interrupt level by writing IE and IL register. All the pending and enable interrupt signals will pass through a five-stage comparator to arbitrate the interrupt source with the highest interrupt level. Finally, the interrupt request signal would be generated and sent to the core if the maximum level is greater than the value of the machine threshold (mth) register. Users can configure the mth register to shield interrupts.

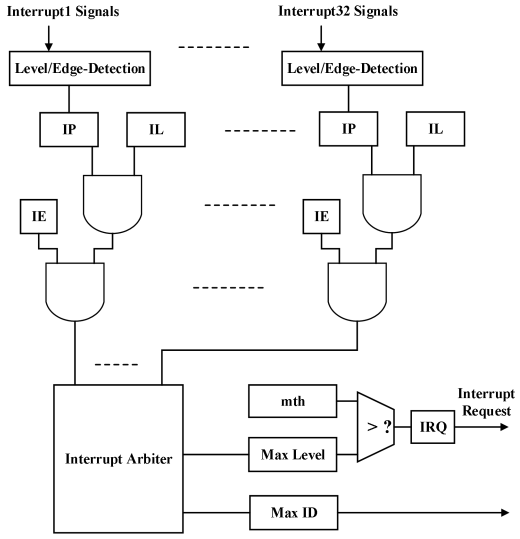


Fig. 4. Logic structure of interrupt request generator

2) Preemption support unit: This unit is about the update of some interrupt-related CSRs to support interrupt preemption. The detailed operation is shown in Table III. The MIE field of mstatus register and the MPIE field of mcause register should not be updated in this design.

TABLE III. UPDATE OF INTERRUPT-RELATED CSRS

CSR	Field Name	Description
mepc	/	Update to current PC value when interrupt happens.
mcause	MPIL	Update to previous interrupt level.
mintstatus	MIL	Update to incoming machine mode interrupt level.

mtvt + (Int.ID*4)
...
mtvt + 0x18
mtvt + 0x14
mtvt + 0x0C
mtvt + 0x08
mtvt + 0x04
mtvt + 0x00

Fig. 5. Interrupt vector table in CLIC vectored mode.

3) Interrupt vector table: The CLIC vectored mode of operation provides the ability to use a vector table for interrupts, as shown in Fig. 5. The processor fetches an 32-bit handler address from the in-memory vector table pointed to by

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on September 29,2026 at 07:10:13 UTC from IEEE Xplore. Restrictions apply.

machine trap vector table (mtvt) register, which defines the base address of the CLIC vector table. Then, the processor sets the PC to this handler address.

C. Extended Design

In order to further reduce the interrupt response time and software overhead in the ISR, we have further extended the design for embedded RISC-V processors.

1) Hardware stacking and unstacking: In order to reduce the software overhead during the interrupt response, we have designed a hardware stacking and unstacking mechanism applied in the interrupt system to save the general-purpose registers used in the ISR, as well as the mepc register and mcause register. For the processor based on RV32-E, there are 10 caller saver registers (ra, t0, t1, t2, a0, a1, a2, a3, a4, a5) which will not be preserved across the call. Therefore, when the processor receives the interrupt request, it automatically pushes 12 registers to the stack memory, and then pops the registers in order when the ISR is finished. The execution flow of hardware stacking and unstacking is shown in Fig. 6.

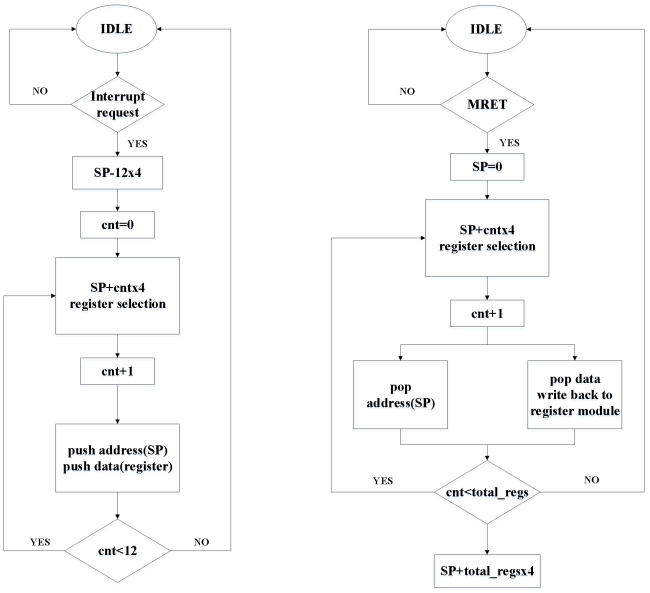


Fig. 6. Execution flow of hardware stacking and unstacking

In the stacking process, the stack pointer (SP) minuses 48 to prepare enough space for 12 registers in the stack memory. Then, ra, t0, t1, t2, a0, a1, a2, a3 a4, a5 register are pushed to the stack from the bottom to the top through the data path of the AHB bus. In the unstacking process, the processor load the data in the stack memory from the top to the bottom and then write the data back to a5, a4, a3, a2, a1, a0, t2, t1, t0, ra registers in order. Finally, the SP adds 48 to restore the original position.

2) Hardware mask for the global interrupt-enable bit: In the RISC-V machine mode, the MIE field of mstatus register is the global interrupt-enable bit. When the interrupt request is accepted by the processor, the value of previous mstatus.MPP and previous interrupt level will be preserved to mcause register. It ensures the restoration of the previous interrupt's status when the interrupt preemption finishes. Therefore, mstatus.MIE need to be cleared to forbid interrupt preemption and prevent the overwriting of mcause register. We have made a hardware mask design for the global interrupt-enable bit, so that there is no need to reset mstatus.MIE by software and protect the previous interrupt's status at the same time. The circuit is shown in Fig. 7.

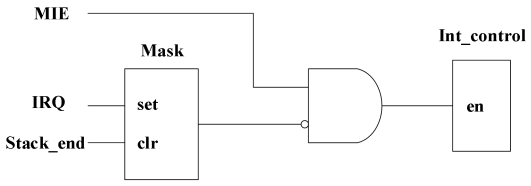


Fig. 7. Mask design for global-enable bit.

The design disables the MIE field when receives the interrupt request. In addition, the mask is cleared once the hardware stacking is finished. The role of the mask is to prevent the preemption by other interrupts before the interrupt status is saved. The MIE field and MPIE field of the mstatus register don't need to be operated by hardware under other conditions in this system.

3) Parallel operation of stacking and vector fetching: Since this system adopts the Harvard architecture, the instruction fetching and the data access can perform at the same time. Meanwhile, we have extended the form of stacking by hardware, so we design the processor to execute stacking and vector fetching in parallel. When the processor receives an interrupt request, the hardware stacking and the vector fetching start at the same time. After stacking, the processor sets the PC to the address fetched in interrupt vector table and jumps to the ISR entry. The interrupt entry sequence of this interrupt system is shown in Fig. 8.

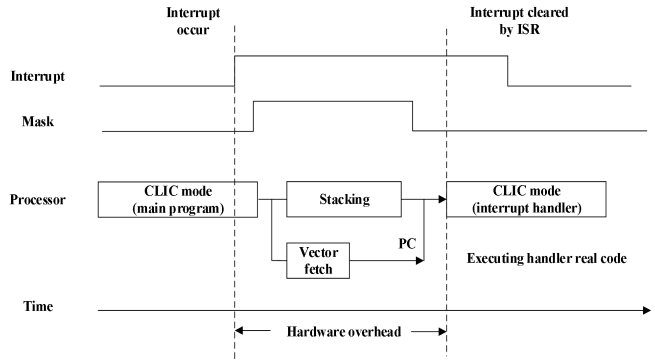


Fig. 8. Interrupt entry sequence of this design

4) Tail chaining: When the processor is handling an interrupt but another interrupt with the same or lower interrupt level takes place, the interrupt will be blocked and pending for response. We have added the tail chaining design to the interrupt system. When the processor has finished executing the current interrupt handler, instead of restoring the registers back from the stack and then pushing them into the stack again, the processor skips the unstacking and stacking steps and handle the pending interrupt directly. Therefore, the timing gap between the two interrupt handlers is considerably

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on September 29,2026 at 07:10:13 UTC from IEEE Xplore. Restrictions apply.

reduced. The behaviour of tail chaining interrupts in this system is shown in Fig. 9.

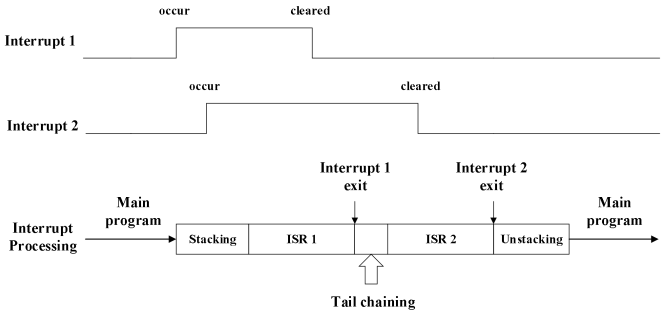


Fig. 9. Tail chaining interrupts behaviour.

IV. Implementation Results and Comparison

The RISC-V processor using the proposed interrupt system is synthesized by Synopsys Design Compiler using the 0.11um CMOS technology. The area report is shown in Table IV.

TABLE IV. Area Report Synthesized by DC

		Area (um^2)	Equivalent gate count
NAND2SIXT1X		3.2856	1
RISC-V processor	**Interrupt system**	13441	4.1k
	Other modules	99783	30.4k

The function of the processor is verified at RTL and gate-level simulations. In order to ensure the correctness of the proposed interrupt system, we translate C-programs into binary programs to validate the interrupt handling and interrupt preemption of the RISC-V processor. FPGA prototyping is carried on Xilinx Kintex-7 (xc7k410t-3fbg676) platform.

We also compare the interrupt response time of the processor applying the proposed interrupt system with that of Cortex-M3 processors. The result is shown in Table V. The interrupt response time of this design is 13 cycles. One cycle for interrupt request sampling, one cycle for pipeline flushing and eleven cycles for ten caller registers' hardware stacking. The interrupt response time is one cycle more than Cortex-M3, because the definition of general-purpose register bank is different in RISC-V and ARM. In this design, there are two more registers during hardware stacking than Cortex-M3.

TABLE V. Interrupt Response Time Comparison with Cortex-M3

Processor	Cortex-M3	This design
Overhead	Hardware overhead	Hardware overhead
Time (cycle)	12	13

V. Conclusion

In this paper, we have studied the interrupt entry sequence of the Cortex-M3 and SiFive-E21, and designed a fast interrupt system based on CLIC extension by referring the interrupt handling mechanism of ARM. The interrupt system supports automatic nested interrupt and there is no additional software overhead in the ISR. We have applied the interrupt system in a three-stage pipeline RISC-V processor based on RV32-E and Harvard architecture. The interrupt response time of this design is comparable to that of a Cortex-M3 processor. The function of the interrupt system is verified by RTL simulations and FPGA prototyping.

Acknowledgment

The authors gratefully acknowledge the help from Xiaohui Xiao, Yangfan Zhou, Chao Fu, and Jie Cao at Hangzhou Vango Technologies, Inc.

This work was supported by NSFC-Zhejiang Joint Fund for the Integration of Industrialization and Informatization under Grant U1709221.

References

[1] L. Zhang, R. Qi, X. Chen, "Talking about the Application of STM32 Microcontroller Interrupt Manager (in Chinese)," Computer Products and Circulation, vol. 10, pp. 115, 2019.

[2] A. Waterman, Y. Lee, D. A. Patterson, and K. Asanovi, "The RISC-V Instruction Set Manual. Volume I: User-Level ISA, Version 2.0," DTIC Document, Tech. Rep., 2014.

[3] RISC-V Core-Local Interrupt Controller (CLIC). [Online]. Available: https://github.com/riscv/riscv-fast-interrupt/blob/master/clic.adoc.

[4] Andrew Waterman and Krste Asanovic, "The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Document Version 20190608-Priv-MSU-Ratified," RISC-V Foundation, 2019.

[5] Joseph Yiu, The Definitive Guide to the ARM Cortex-M3. Elsevier, 2015.

[6] SiFive E21 Core Complex Manual. [Online]. Available: https://sifive.cdn.prismic.io/sifive/7c22c2ec-8af4-4b6c-a5fe-9327d91e7808_e21_core_complex_manual_21G1.pdf

[7] Nuclei ISA SPEC. [Online]. Available: https://doc.nucleisys.com/nuclei_spec.

[8] J. H. Kong, L. -. Ang, K. P. Seng, "Minimal Instruction Set AES Processor using Harvard Architecture," 2010 3rd International Conference on Computer Science and Information Technology, 2010, pp. 65-69.

[9] Y. Wang, N. Tan, "An Application-Specific Microprocessor for Energy Metering Based on RISC-V," 2019 International Conference on IC Design and Technology (ICICDT), 2019, pp. 1-4.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on September 29,2026 at 07:10:13 UTC from IEEE Xplore. Restrictions apply.